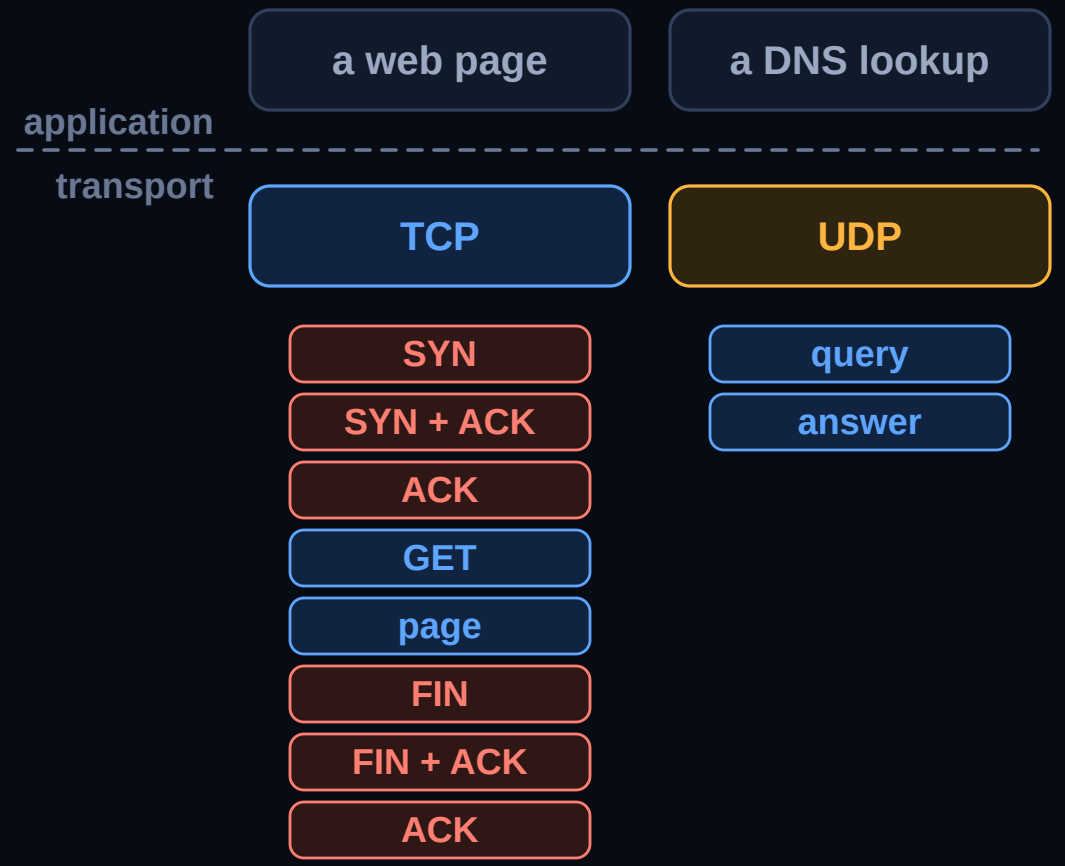
The Real TCP and UDP

The last lectures built the mechanisms. This one looks at the real protocols that carry them: how TCP opens, runs and closes a connection, what UDP leaves out, and why the newest transport, QUIC, is built on UDP.

Yaxiong Xie



Two real transport protocols.

One keeps a connection, with a beginning and an end; the other keeps nothing.

What has this chapter built so far?

reliable transfer

- checksum
- ACKs and sequence numbers
- timers, adaptive timeout
- windows: Go-Back-N, selective repeat
- duplicate ACKs, fast retransmit

congestion control

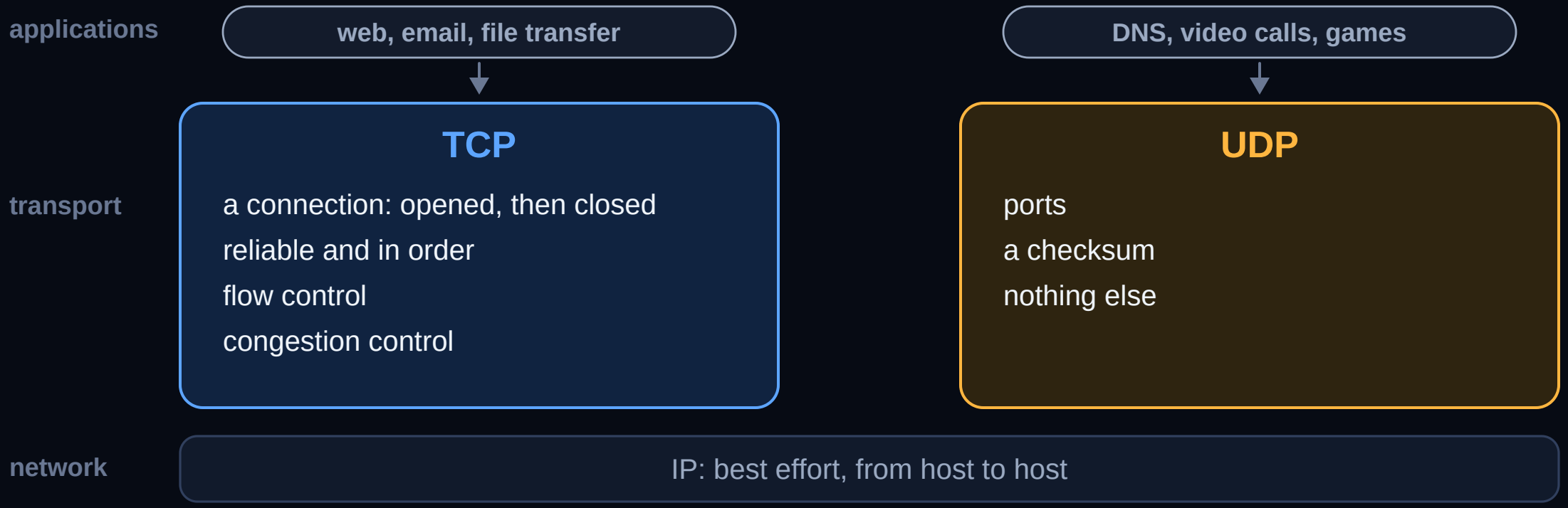
- cwnd
- AIMD
- slow start
- ECN

what those lectures assumed

- A and B already connected, never closed
- packets numbered from 0
- B always had room

We built the pieces; a real protocol cannot make those three assumptions.

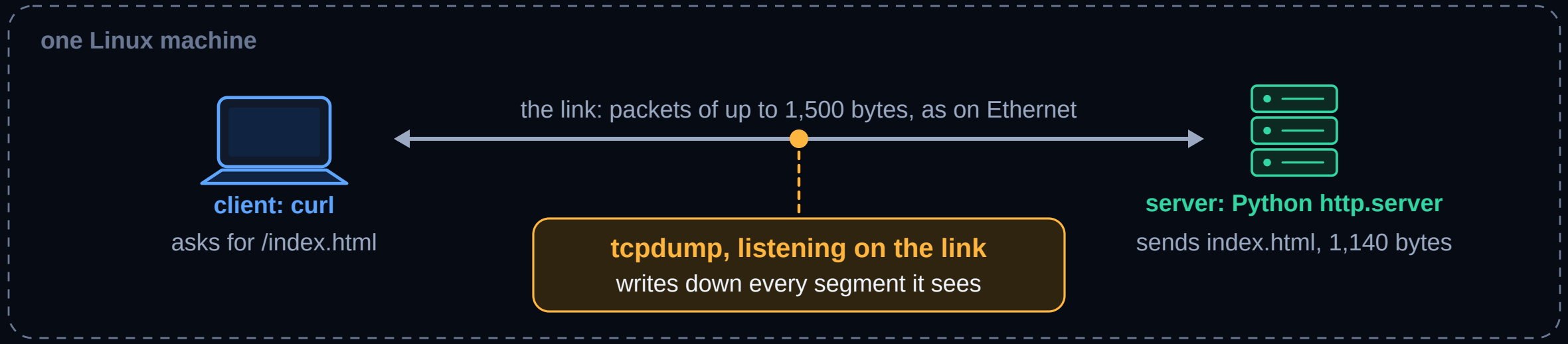
Which real protocols carry these mechanisms?



This lecture: the real TCP and UDP, and QUIC, TCP rebuilt over UDP: 40.8% of websites already serve pages over it.

TCP carries every mechanism of this chapter; UDP carries almost none.

How do we watch a real TCP connection?



server	`python3 -m http.server 8080`	serves this folder
client	`curl --http1.1 http://127.0.0.1:8080/index.html`	fetches the page once
recorder	`tcpdump -i lo 'tcp port 8080'`	every TCP segment

Fetch one small page, and record every segment that crosses the link.

What does a real TCP connection look like?

drawn in this chapter never drawn what tcpdump recorded: 12 segments, one per row

#	from → to	flags	seq	ack	len	win	options, payload
1	client → server	SYN	0		0	64240	mss 1460 · sackOK · wscale 10
2	server → client	SYN, ACK	0	1	0	65160	mss 1460 · sackOK · wscale 10
3	client → server	ACK	1	1	0	63	
4	client → server	PSH, ACK	1	1	87	63	GET /index.html
5	server → client	ACK	1	88	0	64	
6	server → client	PSH, ACK	1	88	188	64	HTTP/1.0 200 OK
7	client → server	ACK	88	189	0	63	
8	server → client	PSH, ACK	189	88	1140	64	(the page)
9	client → server	ACK	88	1329	0	66	
10	server → client	FIN, ACK	1329	88	0	64	
11	client → server	FIN, ACK	88	1330	0	66	
12	server → client	ACK	1330	89	0	64	

Every line carries fields this chapter has never drawn.

How many of these segments carry the page?

#	from → to	flags	len	payload
1	client → server	SYN	0	
2	server → client	SYN, ACK	0	
3	client → server	ACK	0	
4	client → server	PSH, ACK	87	GET /index.html
5	server → client	ACK	0	
6	server → client	PSH, ACK	188	HTTP/1.0 200 OK
7	client → server	ACK	0	
8	server → client	PSH, ACK	1140	(the page)
9	client → server	ACK	0	
10	server → client	FIN, ACK	0	
11	client → server	FIN, ACK	0	
12	server → client	ACK	0	

opening the connection
3 segments, no data

the request and the page
6 segments

closing the connection
3 segments, no data

Half only open or close: a TCP connection has a beginning and an end.

What does a real UDP exchange look like?

Same machine, same recorder, another application: the client asks a DNS server for www.example.com.

#	from → to	ports	len	what it carries
1	client → server	51000 → 53	33	query: www.example.com?
2	server → client	53 → 51000	49	answer: 192.0.2.10

✕ no opening ✕ no ACK ✕ no closing

If the answer is lost, the client simply asks again.

The DNS server is a few lines of Python (dnslib) on port 53; 192.0.2.10 is an address reserved for examples.

UDP sends a datagram and is done: no connection to open or close.

Six questions

the simplification it removes

01	How does a TCP connection begin?	already connected, from 0	Opening
02	How does a TCP connection end?	never closed	Closing
03	What if the receiver cannot keep up?	always had room	Flow control
04	What if an application does not want all of this?		UDP
05	Why can’t TCP just be fixed?		Changing TCP
06	What if transport is rebuilt above UDP?		QUIC

01 to 03 are the real TCP, 04 is UDP, 05 and 06 are why QUIC rebuilt TCP on UDP.

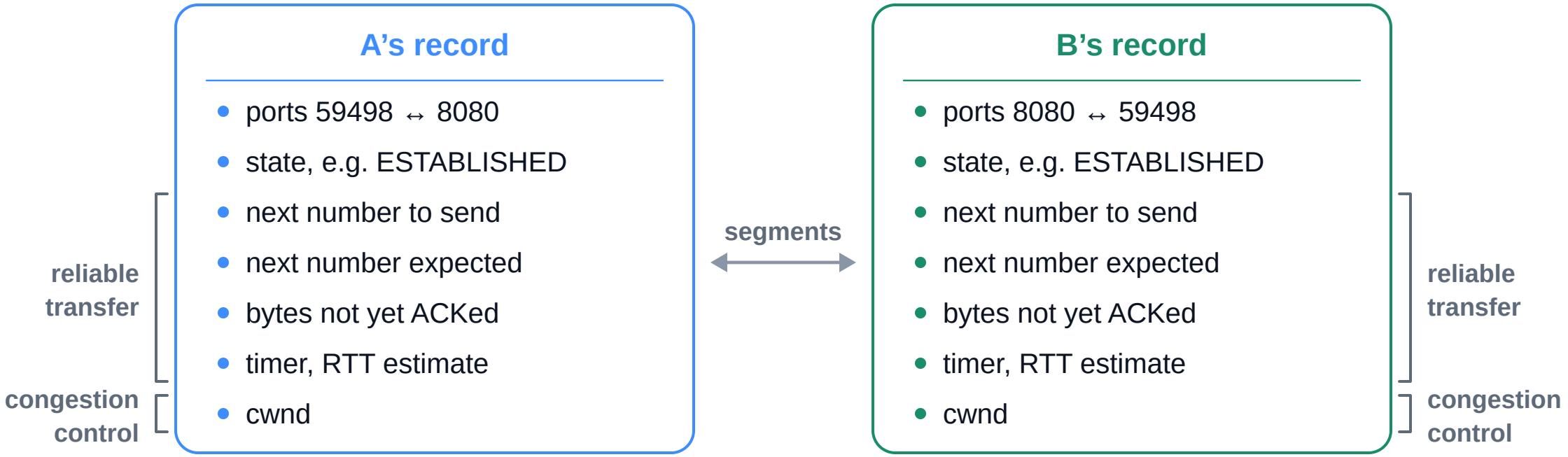
How does a TCP connection begin?

01

Opening a connection

In the trace, three segments went by before any data. What must two hosts set up before the first byte, and why does it take three segments?

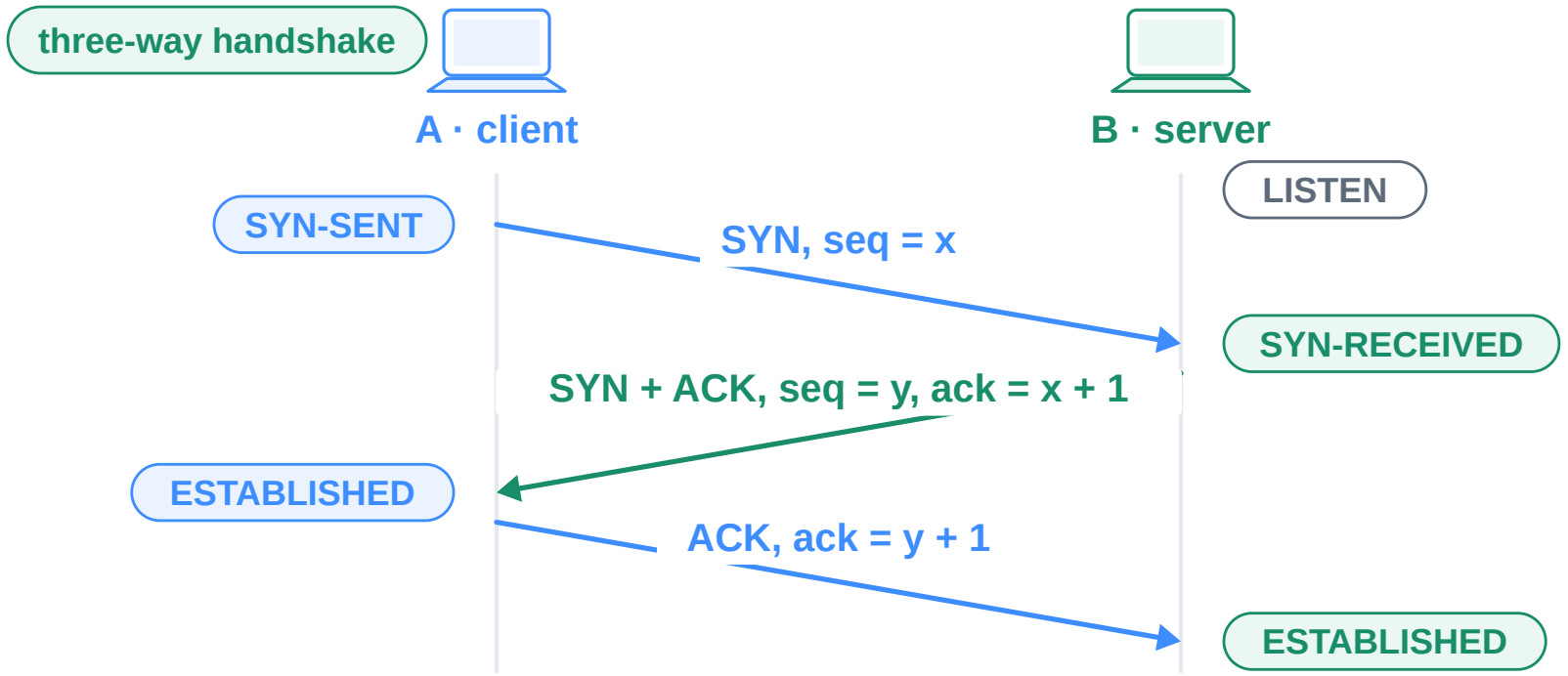
What is a TCP connection?



Routers keep no record of it; NATs and firewalls on the path do. RFC 9293 calls the record the TCB.

A connection is a record kept at both ends: opening creates it, closing frees it.

How does TCP open a connection?



what each segment says

SYN, seq = x
let's connect; I start at x

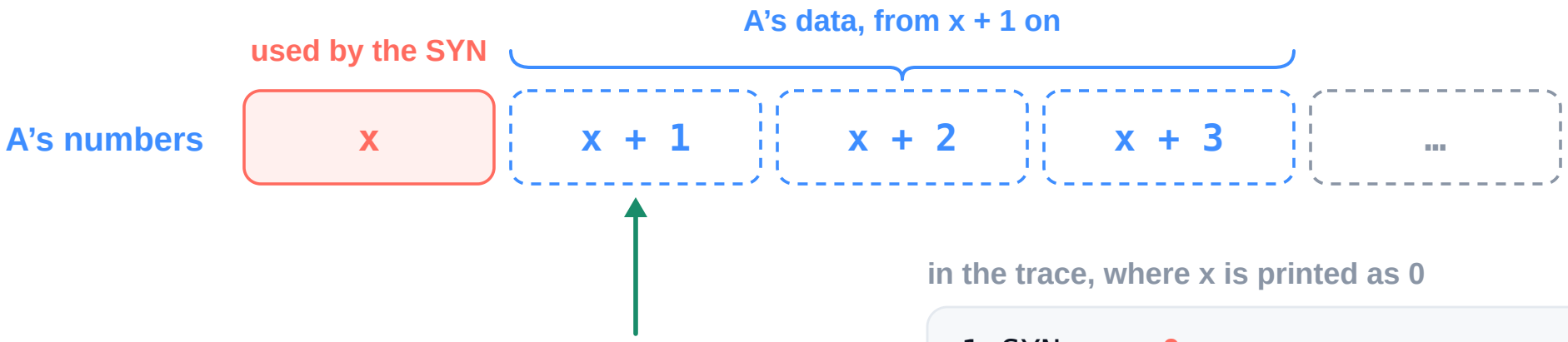
SYN + ACK, ack = x + 1
got x, next I expect x + 1; I start at y

ACK, ack = y + 1
got y, next I expect y + 1

Each ACK answers the other side's number plus one: seq = x is answered by ack = x + 1.

TCP opens with a three-way handshake: each side's number is sent and acknowledged.

What does ack = x + 1 mean?

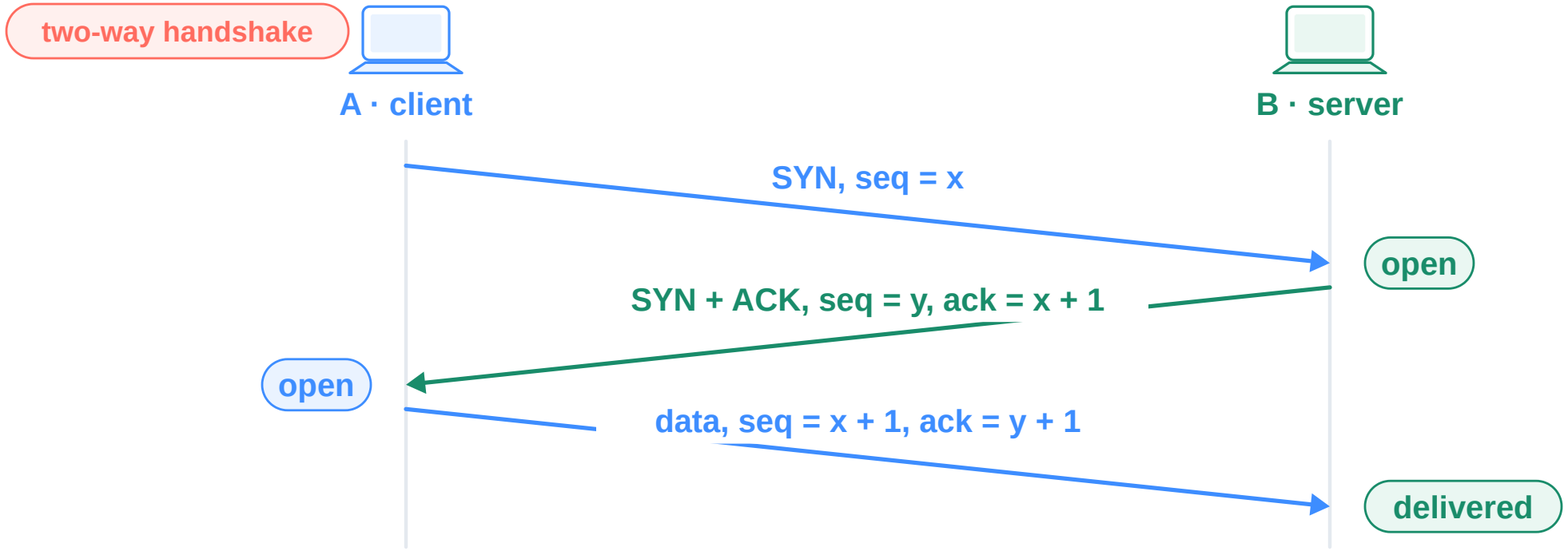


in the trace, where x is printed as 0

```
1 SYN seq 0
2 SYN, ACK ack 1
4 GET seq 1
```

An ACK names the next number expected; the SYN used up x, so B asks for x + 1.

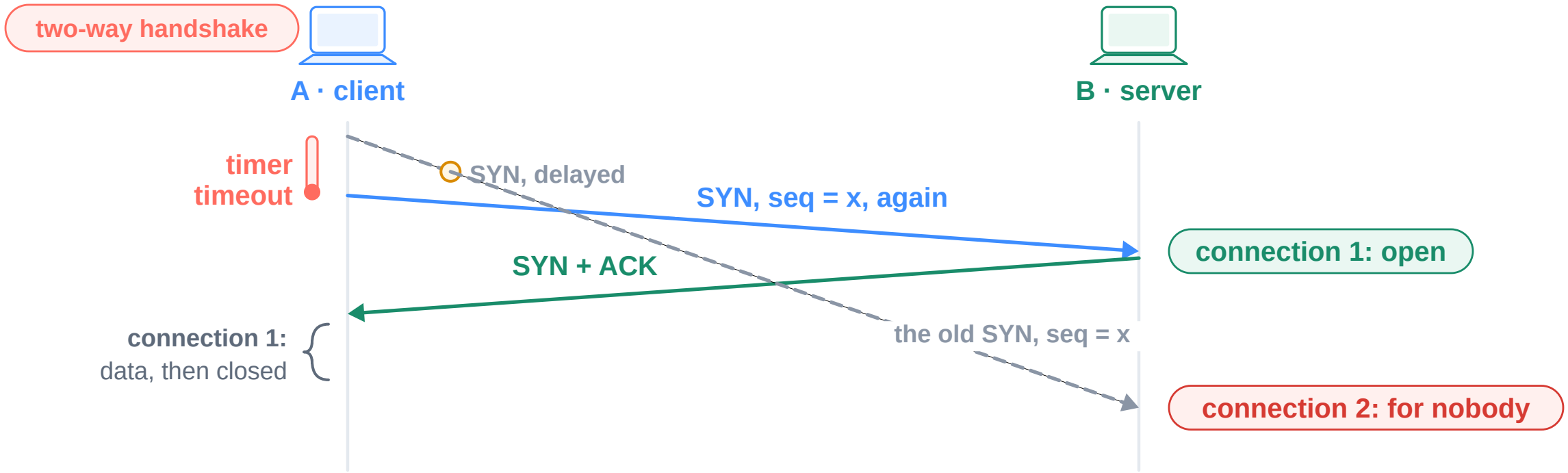
Would a two-way handshake be enough?



Drop the third segment: B opens as soon as the SYN arrives, and A's first data still acknowledges y.

When nothing goes wrong, a two-way handshake looks like enough.

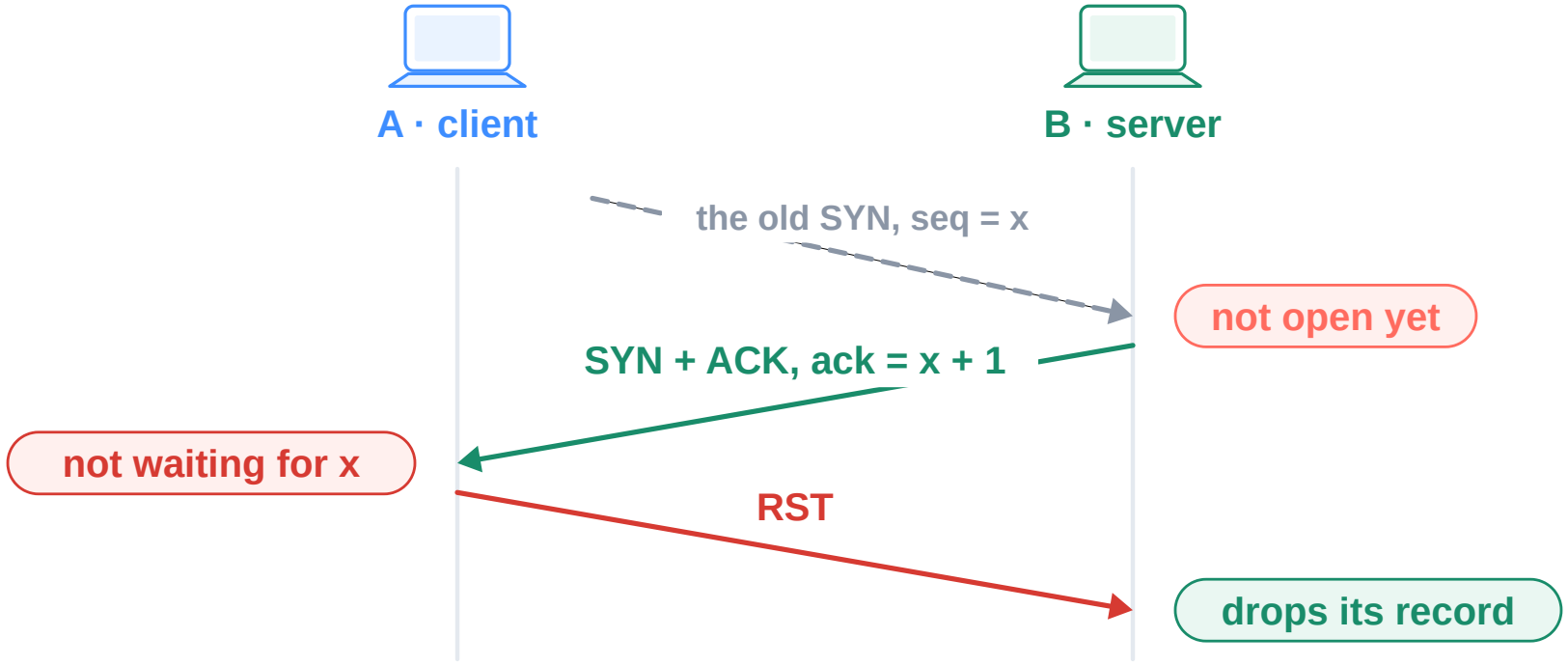
What if an old SYN arrives?



A asked once. Its first SYN was only delayed, so the timeout sent a second copy, and each copy opens a connection at B.

With a two-way handshake, one request from A opens two connections at B.

How does the third segment fix it?



what each segment says

old SYN, seq = x
let's connect; I start at x

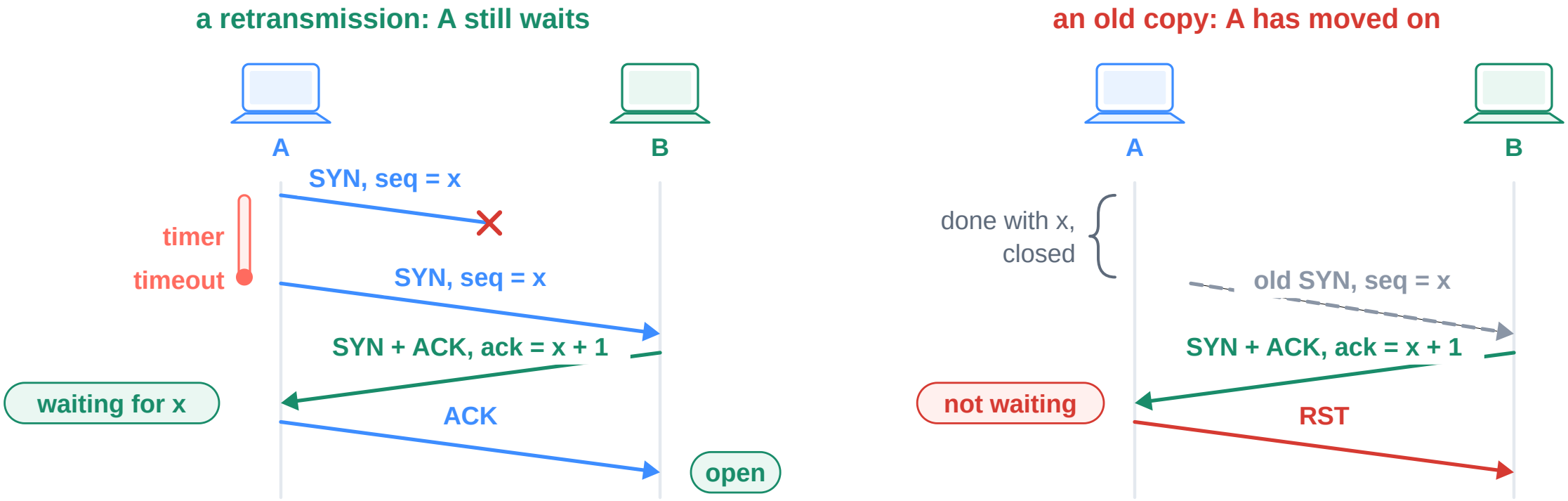
SYN + ACK, ack = x + 1
got x, next I expect x + 1

RST, short for reset
no such connection here: drop it

RFC 9293: "The principal reason for the three-way handshake is to prevent old duplicate connection initiations from causing confusion."

B opens only on A's third segment, so A, the side that knows, can refuse with an RST.

Is a retransmitted SYN an old one?

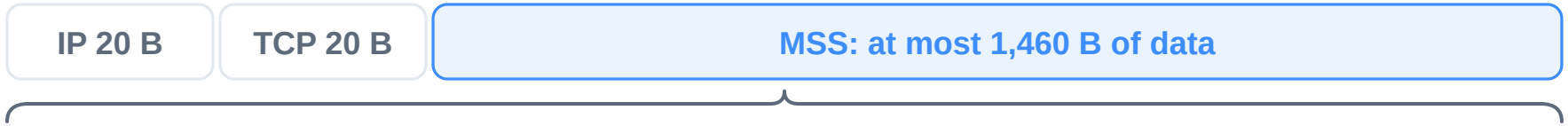


Both copies carry the same x, and B cannot tell them apart. A can: it knows which SYN it is still waiting for.

A completes the SYN it is still waiting for, and refuses a copy it has moved past.

What else does the handshake agree on?

	A's SYN	B's SYN + ACK	what it sets
starting number	x = 2,153,146,557	y = 1,006,549,976	where each side's numbers begin
MSS	1,460	1,460	the most data per segment each side takes
SACK	permitted	permitted	selective ACKs, used if both SYNs offer it

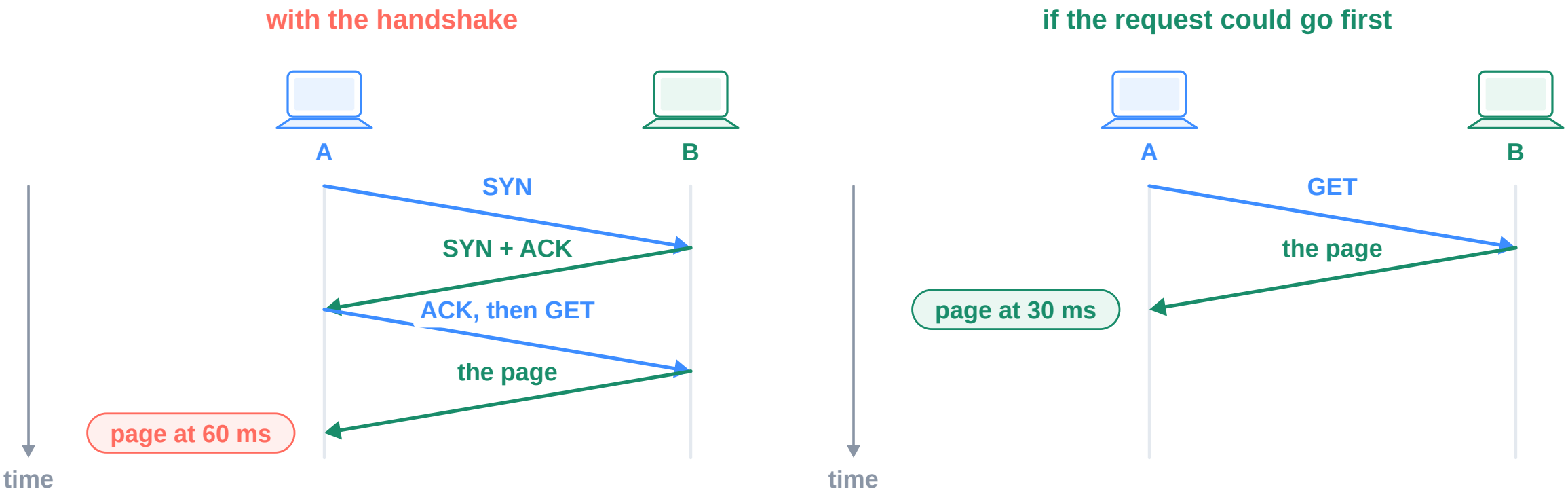


1,500 B: the largest packet the link carries, its MTU

The trace printed x and y as 0: tools subtract them to keep the numbers short.

Each SYN carries its sender's values; an option is used only if both SYNs carry it.

What does the handshake cost?



RTT = 30 ms, the chapter's path. The GET leaves right behind the third segment, as it did in the trace.

Every new connection waits one round trip before its first request.

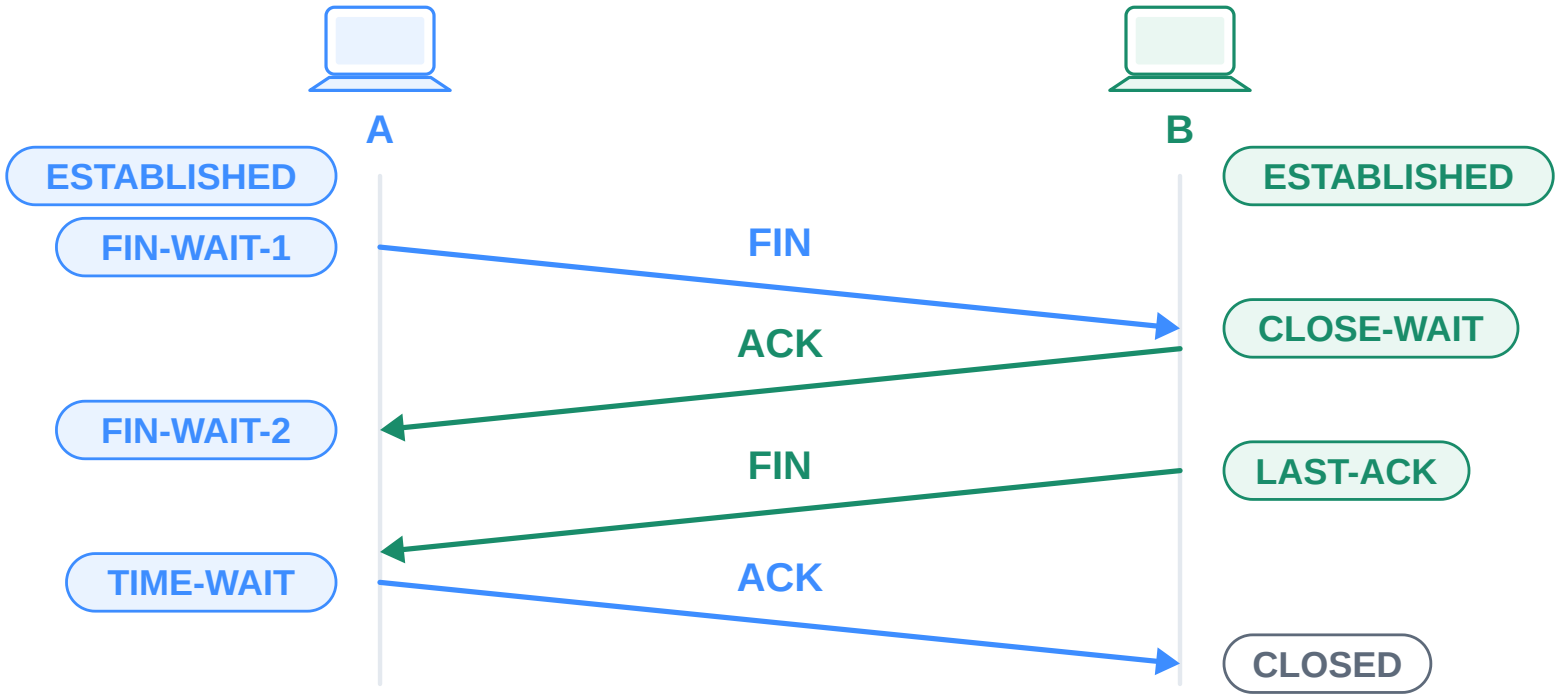
How does a TCP connection end?

02

Closing a connection

Closing frees the record at each end. Before it does, each end must be sure the other has sent its last byte.

How does TCP close a connection?



what each segment says

FIN: I have no more to send.

ACK: I have your FIN.

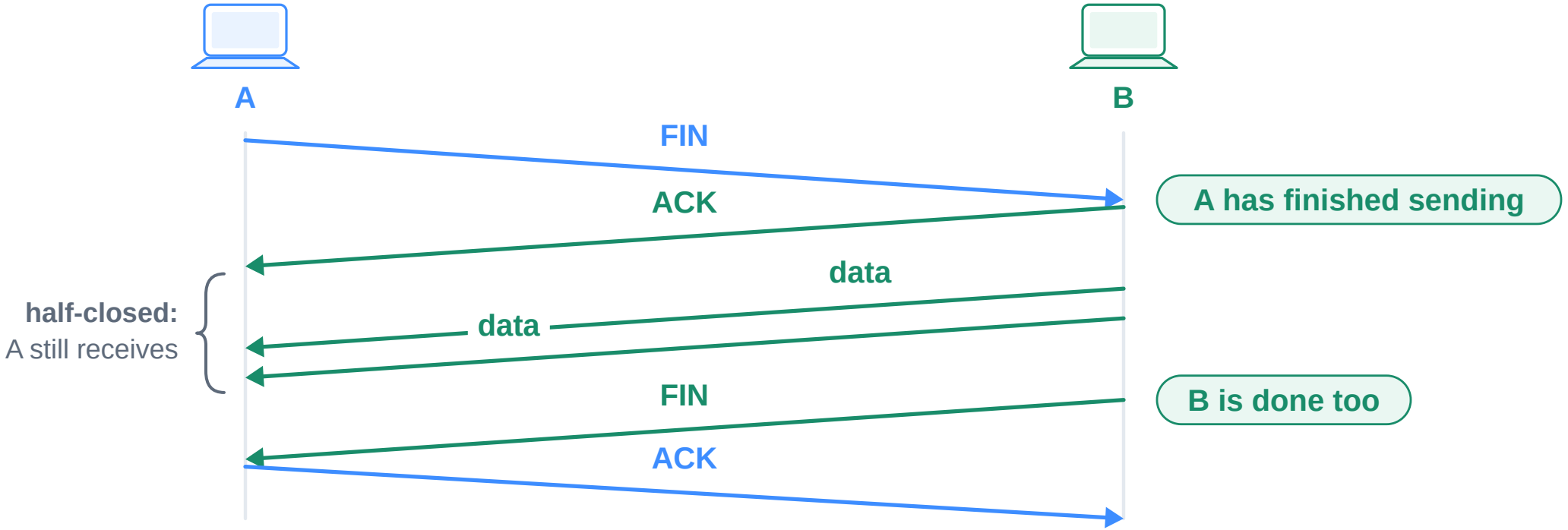
FIN: I have no more either.

ACK: I have your FIN.

The chips are RFC 9293's states. In TIME-WAIT, A waits in case its last ACK is lost and B sends its FIN again.

Each side sends a FIN when it is done sending, and the other acknowledges it.

Why four segments, when opening took three?



If B is done too when A's FIN arrives, its FIN rides with its ACK, as its SYN did: the trace closed in three segments.

B acknowledges A's FIN at once, but sends its own FIN only when it is done.

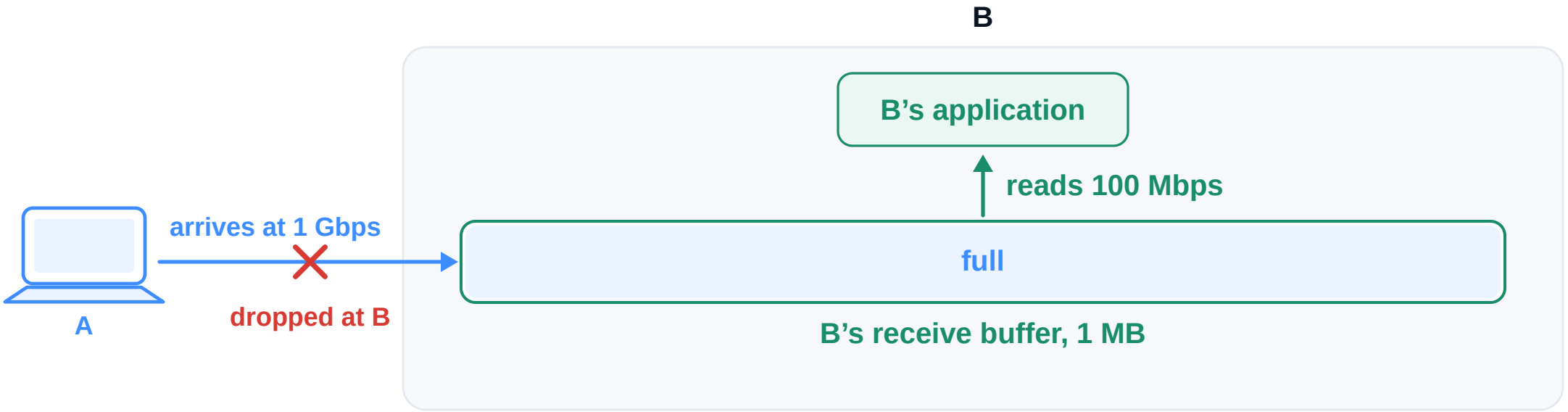
What if the receiver cannot keep up?

03

Flow control

The congestion-control lectures assumed that B always had room. Here B’s application reads slowly, its buffer fills, and A needs a second limit.

What if B’s application reads slowly?

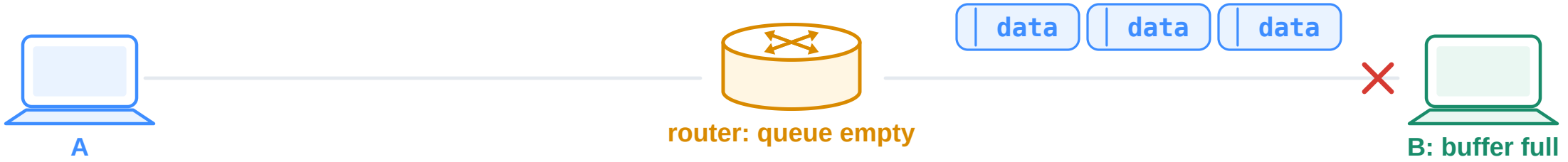


8 Mb ÷ (1 Gbps − 100 Mbps) = 8.9 ms

One key press plays the fill in slow motion: 8.9 ms shown as 4 seconds. 1 MB is 8 Mb; the buffer gains 900 Mbps.

B’s buffer fills in 8.9 ms; every packet after that is dropped at B.

Why not let the loss tell A?

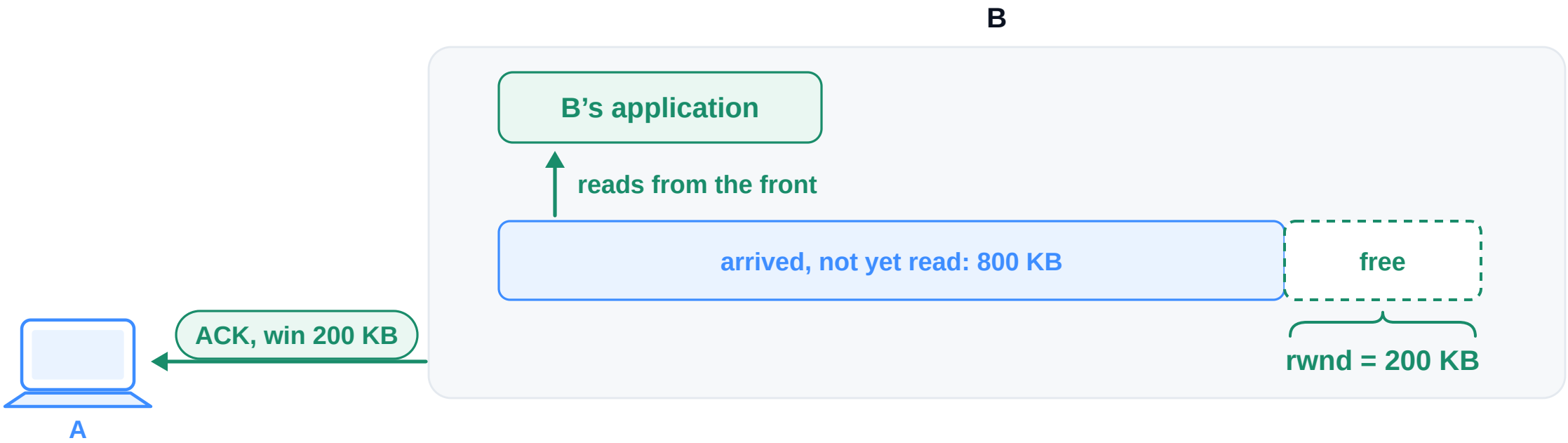


1. **In the round trip before A hears, 3.75 MB arrive** — B can free only 0.375 MB: 3.4 MB are dropped
2. **A reads the loss as congestion and halves cwnd** — while the router's queue is empty

1 Gbps × 30 ms = 3.75 MB in flight; the application reads 100 Mbps × 30 ms = 0.375 MB in that time.

Loss at the receiver wastes a round trip of data and is read as congestion.

What if every ACK says how much room is left?



rwnd = buffer − (**received − read**) = 1 MB − 800 KB = **200 KB**

Every ACK carries rwnd, the free space left in B's buffer.

How does A combine this with cwnd?

what the network allows

cwnd = 2,500 packets = 3.75 MB

what B can hold



rwnd = 200 KB



window = min(cwnd, rwnd) = 200 KB

rate ≤ window ÷ RTT = 200 KB ÷ 30 ms = **53 Mbps**

The congestion-control lectures assumed B always had room, so the window there was cwnd alone.

A's window is the smaller of what the network allows and what B can hold.

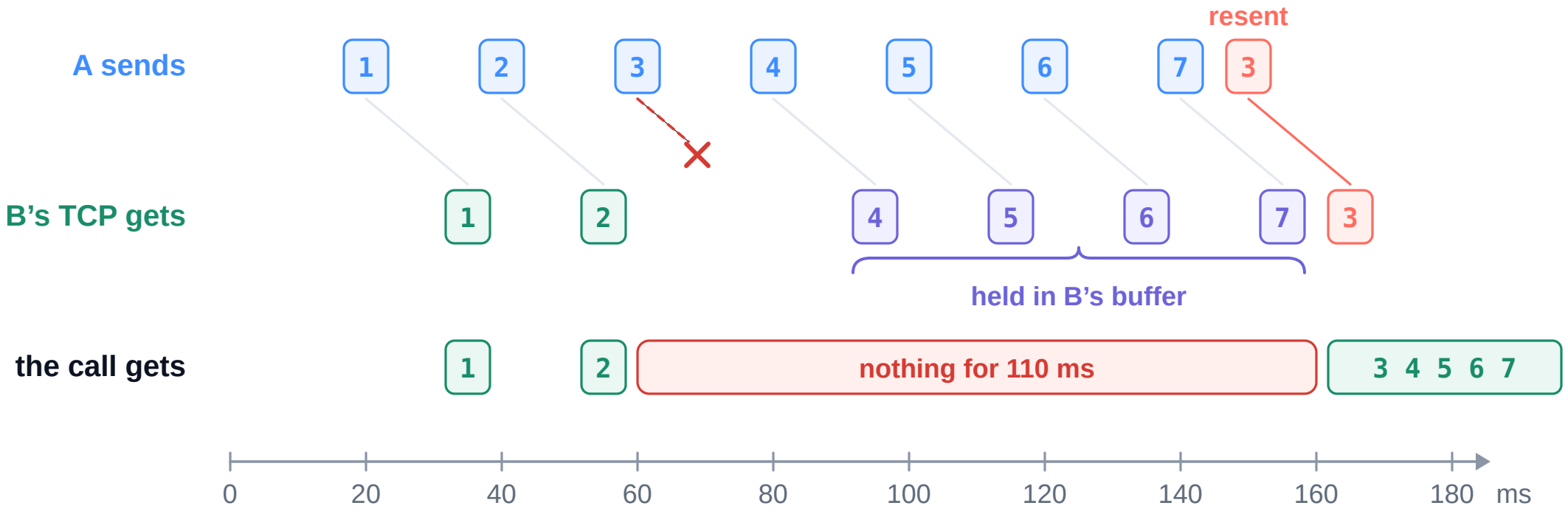
What if an application does not want all of this?

04

UDP

Every guarantee TCP gives is paid for in waiting. Here is an application that cannot wait, and what is left when the guarantees go.

What does one lost packet do to a live call?



A frame every 20 ms, RTT 30 ms; three duplicate ACKs make A resend 3. One key press plays it 50 times slower.

Head-of-line blocking: frames that arrived in time wait for the one that was lost.

1 Opening | 2 Closing | 3 Flow control | 4 UDP | 5 Changing TCP | 6 QUIC

What does each TCP guarantee cost?

Opening	**One round trip before the first byte** paid by every new connection, before any data
In order	**A loss holds back everything behind it** head-of-line blocking, as in the call
Reliable	**Data is resent even when it is too late to use** a frame that arrives 90 ms late is not worth playing
Rate	**TCP’s congestion control sets the sending rate** not the application, which may know its media better

Every guarantee is paid for in waiting, and some applications cannot wait.

What is left without them?

the whole UDP header

8 bytes

Source port	Destination port
Length	Checksum

ports: which socket gets it
checksum: bit errors

each send() is one datagram, delivered whole or not at all

what UDP leaves out

- ✗ no handshake
- ✗ no sequence numbers or ACKs
- ✗ no retransmission
- ✗ no rwnd, no cwnd

UDP adds ports and a checksum to IP; everything else is the application's.

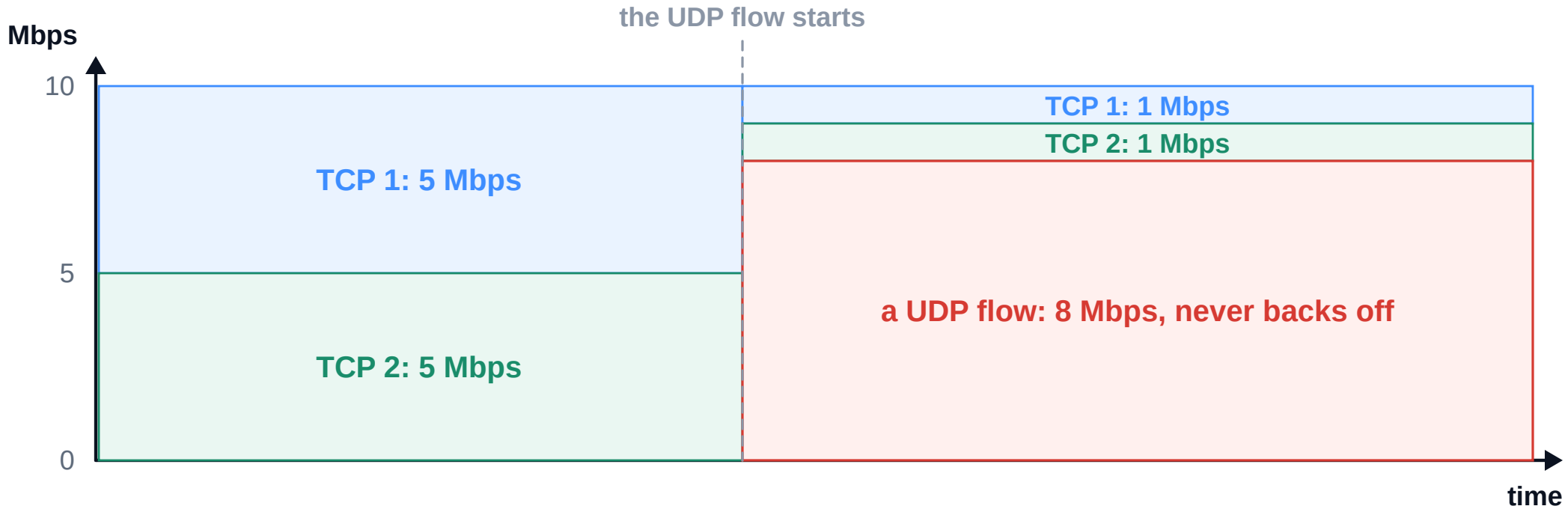
What do applications on UDP add back?

application	what it adds back	what it leaves out
DNS lookup	a timeout and a retry	a connection, order
live call (RTP)	numbers, timestamps, its own rate	resending late frames
online game	the latest state in every packet	resending old state
clock sync (NTP)	timestamps in each reply	everything else

RTP (RFC 3550) numbers and timestamps every packet, so a receiver can reorder and time them without TCP.

An application on UDP rebuilds only the mechanisms it needs.

Can a UDP sender send as fast as it likes?



A model of one 10 Mbps bottleneck. RFC 8085: an application that does not use congestion-controlled transport SHOULD control its rate.

UDP never slows a sender down, so the application must slow itself down.

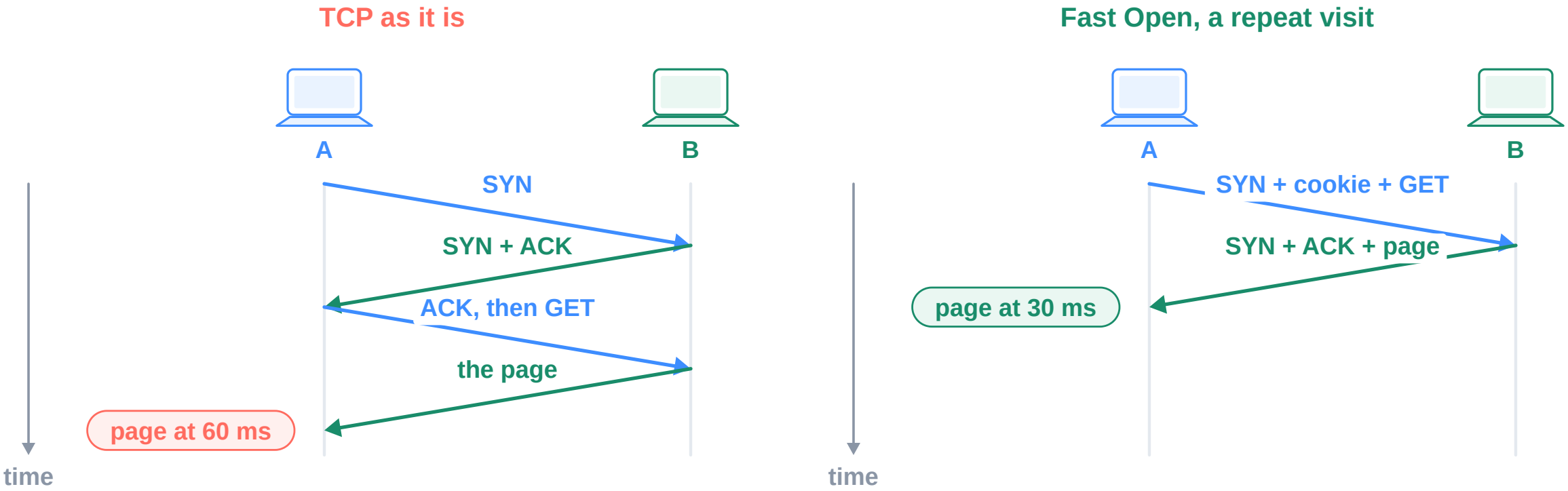
Why can’t TCP just be fixed?

05

Changing TCP

TCP’s costs were measured long ago, and fixes were designed and standardized. Here we follow one of them and ask why it never reached most users.

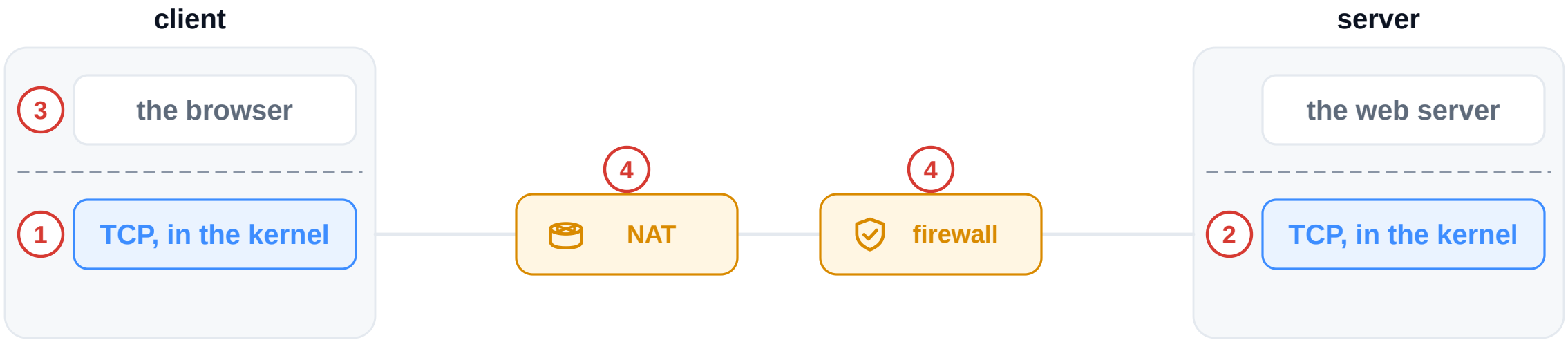
Can the request ride in the SYN?



Proposed by Google in 2011, RFC 7413 in 2014. The cookie, from an earlier connection, shows the client was at this address before.

TCP Fast Open saves one round trip on every repeat connection.

Who has to change for it to work?

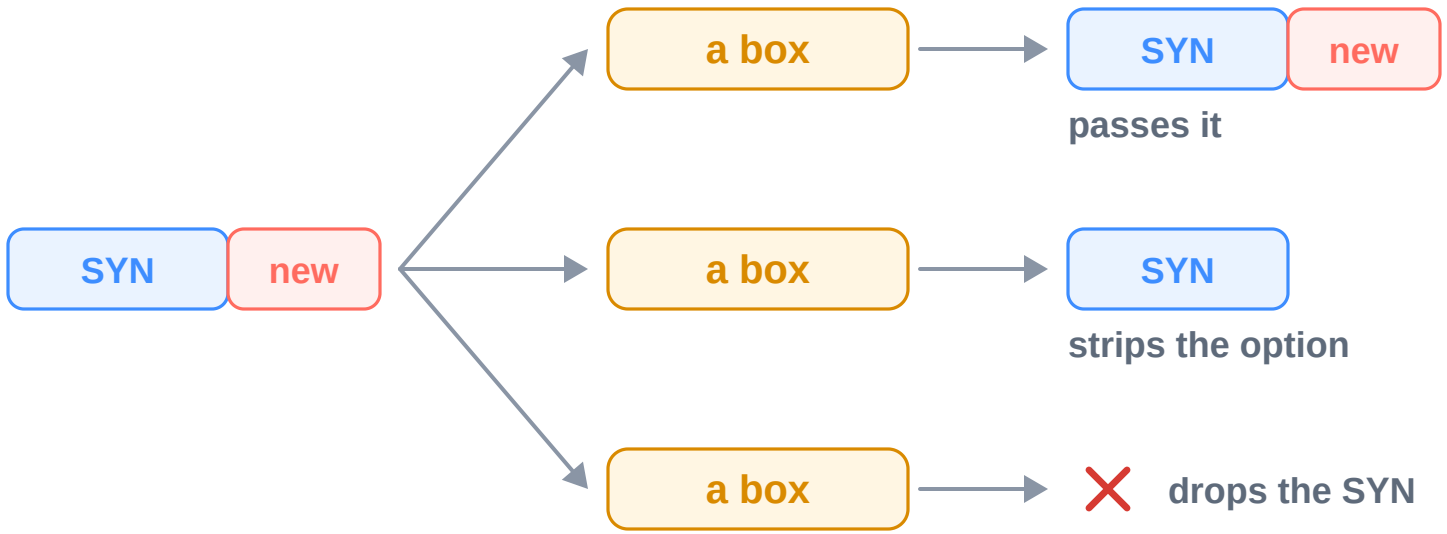


1. the client's kernel: puts data in its SYN
2. the server's kernel: answers with data
3. the application: asks for it with a new call
4. every box on the path: lets that SYN through

Changing TCP needs both kernels, the application and every box between.

1 Opening | 2 Closing | 3 Flow control | 4 UDP | 5 Changing TCP | 6 QUIC

What do boxes on the path do to a new option?



port 80, 142 networks (2011)

an unknown SYN option

86% untouched

14% removed

sequence numbers

82% untouched

18% rewritten

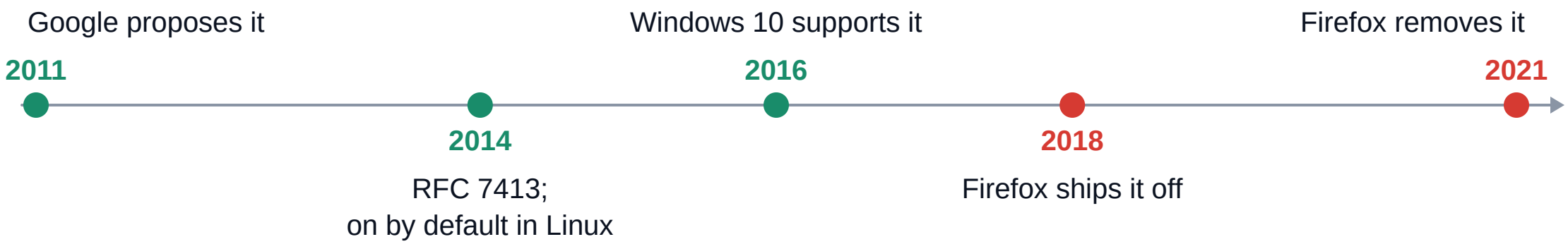
at least 25% of paths interfere
with TCP beyond firewalling

Honda et al., IMC 2011: 142 networks in 24 countries. A protocol that can no longer change this way is called ossified.

A box that parses TCP strips what it does not know: the format freezes.

1 Opening | 2 Closing | 3 Flow control | 4 UDP | 5 Changing TCP | 6 QUIC

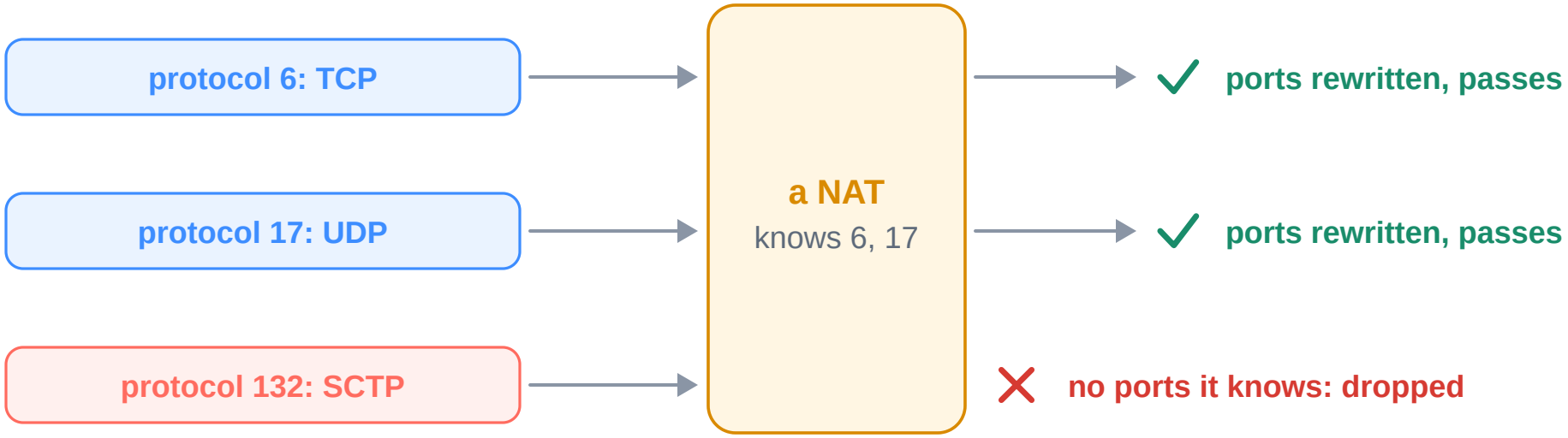
What happened to TCP Fast Open?



why: some boxes on the path drop a SYN that carries data,
so a client had to notice and try again without it

A standard one-round-trip fix, in every major kernel, ended up switched off.

Could a new transport run beside TCP instead?



SCTP (2000): several streams per connection, no head-of-line blocking between them

SCTP lives on in telephone networks; on the web, WebRTC runs it inside DTLS inside UDP (RFC 9260, RFC 8261, RFC 8831).

A new transport crosses today’s internet only if it looks like TCP or UDP.

Then what can still change?

one host can change it alone

- ✓ **how fast it sends: congestion control**
- ✓ Linux loads the algorithm as a module
- ✓ since Linux 5.6, even an eBPF program

both ends and every box must agree

- ✗ **a new handshake: Fast Open**
- ✗ a new option or header field
- ✗ a new protocol number: SCTP

Google moved its servers to a new algorithm, BBR, changing only the sender

BBR: Cardwell et al., ACM Queue 2016. Congestion control written in eBPF (struct_ops) arrived in Linux 5.6.

A sender can change how it sends; nobody can change what TCP looks like.

What if transport is rebuilt above UDP?

06

QUIC

A new transport must look like UDP to cross the internet, and must live outside the kernel to keep changing. QUIC does both.

What does the same page look like over HTTP/3?

Same machine, same page, now over HTTP/3: an aioquic client and server on port 4433, recorded by tcpdump.

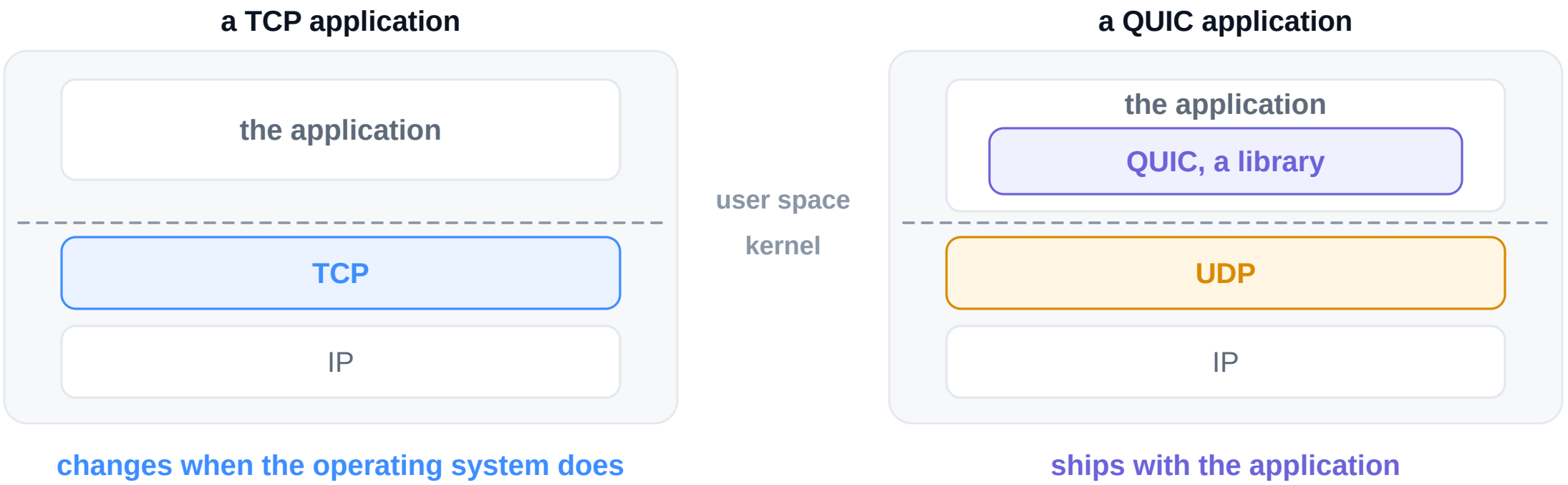
#	from → to	UDP bytes	a box on the path can read	inside, encrypted
1	client → server	1,200	Initial · version · connection IDs	ClientHello, padding
2	server → client	1,200	Initial + Handshake · connection IDs	ServerHello, certificate
3	client → server	1,200	Initial + Handshake · connection IDs	Finished, GET /index.html
4	server → client	1,200	connection ID	200 OK, most of the page
5	server → client	232	connection ID	the rest of the page
6	server → client	33	connection ID	ACK
7	client → server	30	connection ID	close

Same page as the TCP trace: 12 readable segments there, 7 mostly encrypted datagrams here

The newest transport rides on UDP and hides almost everything from the path.

Where does QUIC run?

PROBLEM TCP lives in the kernel, so every change waits for every operating system.



changes when the operating system does

ships with the application

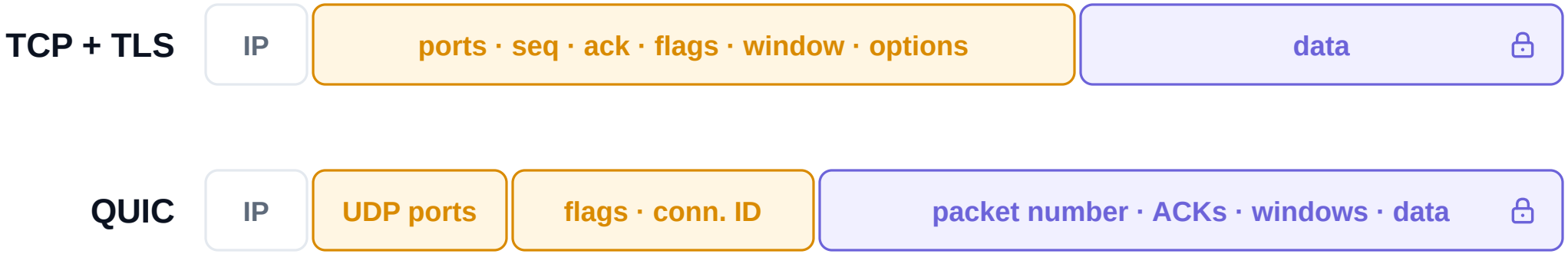
QUIC is a library in the application; the kernel and the path see only UDP.

1 Opening | 2 Closing | 3 Flow control | 4 UDP | 5 Changing TCP | 6 QUIC

Why encrypt the transport header?

PROBLEM **Boxes on the path read TCP's header and break what they do not know.**

a box on the path can read it encrypted



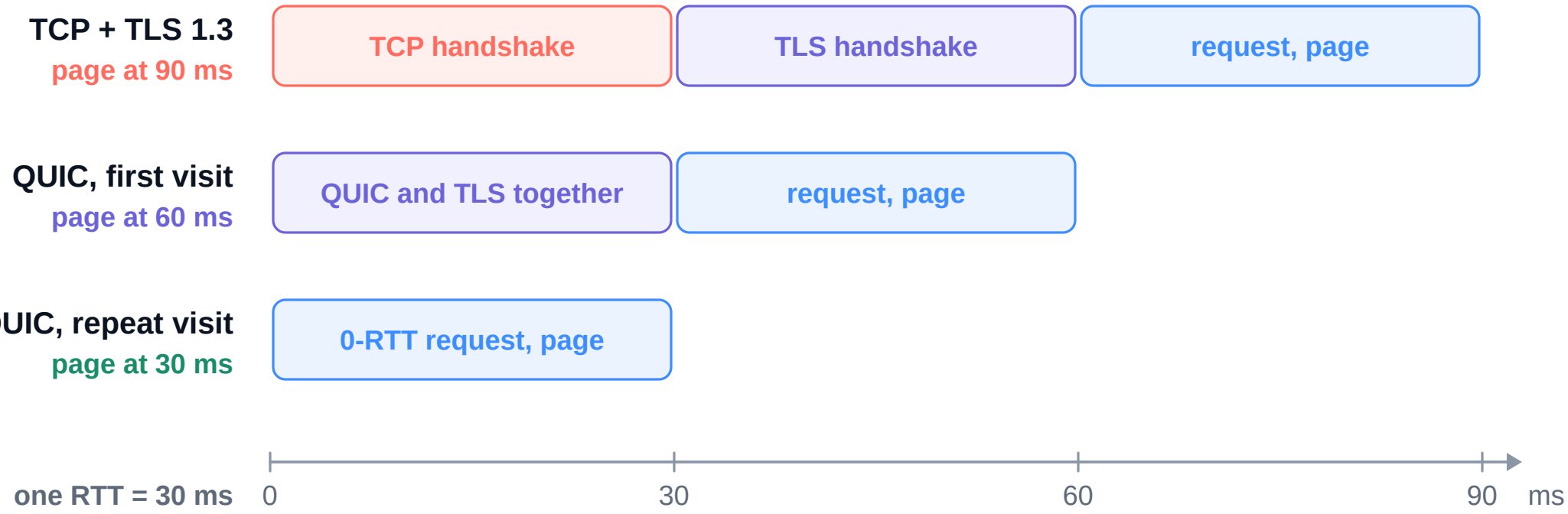
what stays readable is kept small on purpose, so QUIC can change the rest

What a box on the path cannot read, it cannot come to depend on.

1 Opening | 2 Closing | 3 Flow control | 4 UDP | 5 Changing TCP | 6 QUIC

How many round trips before the request?

PROBLEM TCP and TLS each take a round trip before the request, and Fast Open was switched off.

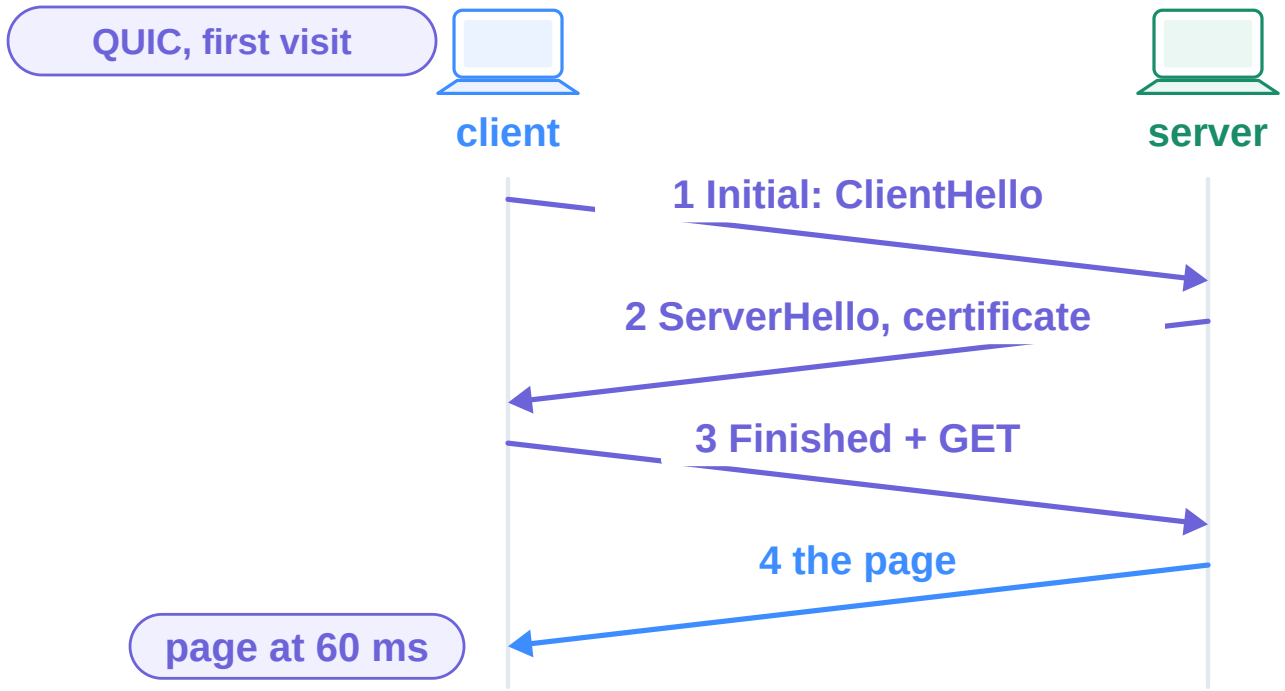


One combined handshake saves a round trip; a repeat visit can send at once.

1 Opening | 2 Closing | 3 Flow control | 4 UDP | 5 Changing TCP | 6 QUIC

Is QUIC’s handshake two-way?

PROBLEM **A two-way handshake would let a delayed copy open a connection for nobody.**



what each datagram says

1 like a SYN
let’s connect; my keys

2 like a SYN + ACK
got it; here are my keys

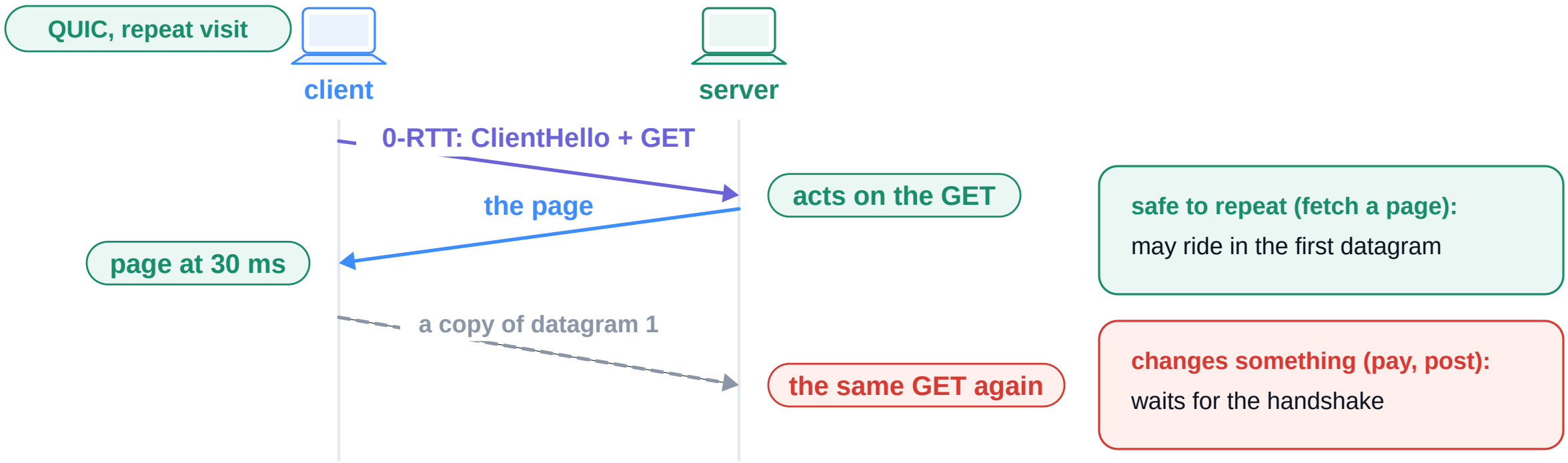
3 like the ACK
got yours; my request too

4 the page
two round trips in all

No: QUIC still opens in three messages; TLS rides inside them, the request on the third.

What if the request rides in the first datagram?

PROBLEM With no handshake first, a request cannot show it is not an old copy.

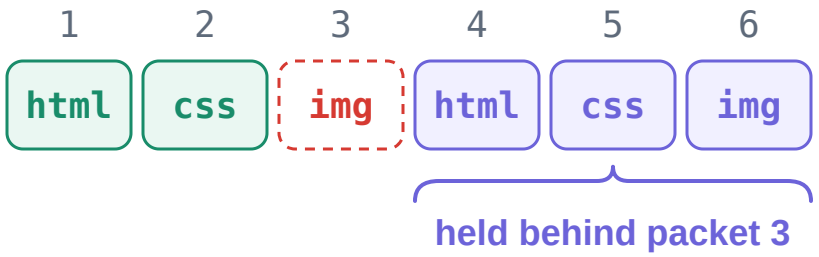


A first-datagram request may be a copy, so 0-RTT carries only requests safe to repeat.

Does one lost packet still stall the page?

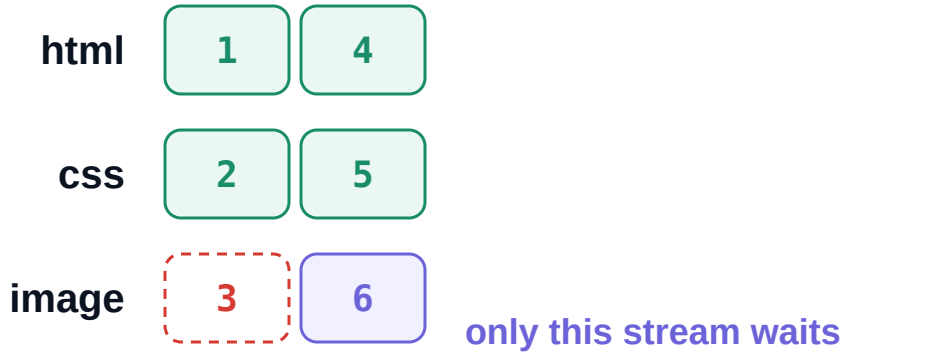
PROBLEM Over TCP, one loss held back every frame of the live call behind it.

TCP: one ordered byte stream



the page gets: 1, 2

QUIC: three streams



the page gets: 1, 2, 4, 5

Each QUIC stream is ordered on its own, so a loss delays only its stream.

1 Opening | 2 Closing | 3 Flow control | 4 UDP | 5 Changing TCP | 6 QUIC

What if the phone changes networks?

PROBLEM TCP names a connection by addresses and ports, and a moving phone gets a new address.

the phone moves from Wi-Fi (198.51.100.20) to cellular (203.0.113.9)

TCP — from 203.0.113.9 → the server looks up (203.0.113.9, 51000, server, 443)

✗ no such connection: reset

QUIC — from 203.0.113.9, ID 7f3a → the server looks up connection ID 7f3a

✓ found: checks the new path, continues

A connection ID names a QUIC connection, so it survives a new address.

What does user space cost?

CPU	**More work per byte than kernel TCP** Google, 2017: 3.5 times TLS/TCP at first, about 2 times after tuning
Trust	**Each application ships its own congestion control** nothing in the network checks it
Visibility	**Operators can no longer read RTTs or losses** one optional spin bit is all that is left
Reach	**Some networks block UDP altogether** 3 to 5% of networks: clients fall back to TCP (RFC 9308)

User space buys freedom to change, paid in CPU, trust, visibility and reach.

Six answers

01	a record at each end, opened by a three-way handshake; A refuses old SYNs	Opening
02	a FIN from each side, and each FIN acknowledged	Closing
03	every ACK carries rwnd; the window is min(cwnd, rwnd)	Flow control
04	ports and a checksum; the rest, rate included, is the application’s	UDP
05	kernels and boxes freeze the wire; only the sender moves	Changing TCP
06	the whole table rebuilt above UDP, encrypted so it can change	QUIC

Each answer is a mechanism of this chapter, packaged into a protocol.

The chapter’s table, with three columns

the defect	the mechanism it forces	TCP	UDP	QUIC
Bits flip inside a packet	**Checksum**	✓	✓	✓
A cannot see what arrived at B	**ACK from the receiver**	✓	—	✓
The feedback itself is damaged	**Sequence number**	✓	—	✓
A packet is lost	**Timer; resend when it expires**	✓	—	✓
Loss looks the same as delay	**Adaptive timeout**	✓	—	✓
The link sits idle	**Window of bandwidth × RTT**	✓	—	✓
One packet in a window is lost	**Go-Back-N or selective repeat**	✓	—	✓
A number is reused too soon	**Bounded window**	✓	—	✓
Loss is detected late	**Duplicate ACKs, selective ACK**	✓	—	✓
The receiver overflows	**rwnd in every ACK**	✓	—	✓
The network overflows	**Congestion window**	✓	—	✓
An old SYN opens a connection	**Three-way handshake**	✓	—	✓

coral rows: added today

UDP keeps one row; TCP keeps all in the kernel, QUIC all in the application.

What to ask of the next transport

Start, stop	**How many round trips before data, and how does it end?**	three-way handshake, FINs, 0-RTT
Delivery	**Reliable? In order? Per stream, or everything together?**	retransmission, head-of-line blocking, streams
Rate	**Who limits it: the network, the receiver, the application?**	cwnd, rwnd, or nobody
Home	**Where does it run, and who must change to change it?**	the kernel, the application, every box on the path

Next chapter: the network layer, home of the routers and boxes in between.